

Degree Project in Technology

First Cycle, 15 Credits

Adaptive Incentive Design for Dynamical Games with Unknown Dynamics: Model-Based and Model-Free Approaches

Jonathan Svensson

Julia Forster

Abstract

This thesis studies incentive design in a dynamic multi-agent system, where agents act according to individual objectives that may differ from the objective of a system-level leader. The problem is investigated using a simplified two-zone thermal building model formulated as a linear-quadratic-Gaussian control problem. The objective is to design incentives that align the agents' Nash equilibrium behavior with the leader's optimal cost.

As a reference, an analytical model-based incentive design method from existing literature is implemented for the case where the system dynamics are fully known. This serves to verify the implementation and provides a benchmark for comparison.

Two data-driven approaches are then considered. First, a recursive least squares (RLS) method is used to estimate the unknown system matrix A, after which incentives are computed using the estimated model. Second, a model-free reinforcement learning approach based on deep deterministic policy gradient (DDPG) is used to learn incentives directly from observed performance.

The results show that the RLS-based method provides stable convergence and achieves low cost error, although it relies on an accurate model estimate. The model-free DDPG approach is able to learn effective incentives, but exhibits slower and less stable convergence and requires more data to reach comparable performance.

Overall, the results highlight a trade-off between model-based and model-free approaches: model-based methods provide superior performance when the system structure is known or can be accurately estimated, while model-free methods remain useful in settings where the system dynamics are unknown or difficult to model.

Sammanfattning

Detta examensarbete behandlar incitamentsdesign i ett dynamiskt multiagentsystem, där agenter agerar utifrån individuella mål som kan skilja sig från målet hos systemets ledare. Problemet studeras med hjälp av en förenklad tvåzons termisk byggnadsmodell, formulerad som ett linjärt-kvadratiskt-gaussiskt (LQG) styrproblem. Syftet är att utforma incitament som får agenternas Nash-jämviktsbeteende att överensstämma med den optimala kostnaden för systemet.

Som referens implementeras en analytisk, modellbaserad metod för incitamentsdesign från befintlig litteratur, i fallet där systemdynamiken är helt känd. Denna används för att verifiera implementationen samt för jämförelse.

Därefter studeras två datadrivna angreppssätt. Först används en metod baserad på rekursiv minsta kvadrat (RLS) för att skatta den okända systemmatrisen A, varefter incitament beräknas utifrån den skattade modellen. Därefter används en modellfri förstärkningsinlärningsmetod baserad på deep deterministic policy gradient (DDPG) för att lära sig incitament direkt från observerad data.

Resultaten visar att den RLS-baserade metoden uppvisar stabil konvergens och uppnår låg kostnadsavvikelse, men är beroende av en noggrann modellskattning. Den modellfria DDPG-metoden kan lära sig effektiva incitament, men uppvisar långsammare och mindre stabil konvergens samt kräver mer data för att nå jämförbar prestanda.

Sammanfattningsvis visar resultaten en avvägning mellan modellbaserade och modellfria metoder: modellbaserade metoder ger bättre prestanda när systemstrukturen är känd eller kan skattas noggrant, medan modellfria metoder är användbara när systemdynamiken är okänd eller svår att modellera.

Acknowledgements

We would like to thank our supervisor Lantian Zhang.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Formulation . . . 1
  - 1.2.1 Multi-agent Dynamical System . . . 1
  - 1.2.2 Agents and the Leader . . . 2
  - 1.2.3 The Incentive Design Problem . . . 3
  - 1.2.4 Problem Setup . . . 3
  - 1.2.5 Thesis Questions . . . 5
- 1.3 Use of AI Tools . . . 5

2 Theoretical Background **5**
- 2.1 Optimal Control Problem . . . 6
- 2.2 Agent Nash Equilibrium . . . 6
- 2.3 Social Optimal Solution . . . 7
- 2.4 Incentive Design when the Model is known . . . 7
- 2.5 Recursive Least Squares . . . 8
- 2.6 Deep Deterministic Policy Gradient . . . 9

3 Methods **9**
- 3.1 Model-based Methods . . . 9
  - 3.1.1 Incentive Design with a known Model . . . 9
  - 3.1.2 Incentive Design with an unknown Model . . . 10
- 3.2 Model-free Reinforcement Learning (DDPG) . . . 12
  - 3.2.1 Actor and Critic . . . 12
  - 3.2.2 Reward Definition . . . 12
  - 3.2.3 Exploration Noise . . . 13
  - 3.2.4 DDPG Updates . . . 13
  - 3.2.5 Network architecture . . . 14
  - 3.2.6 Hyperparameters . . . 14

4 Results **15**
- 4.1 Incentive Design when Model is known . . . 15
- 4.2 Incentive Design when Model is unknown . . . 16
- 4.3 Model-free Incentive Design . . . 18

5 Analysis **20**
- 5.1 Model Verification . . . 20
- 5.2 Convergence Behavior of the RLS Method . . . 21
- 5.3 Robustness of the RLS Method . . . 21
- 5.4 Convergence Behavior of Model-free Approach . . . 21
- 5.5 Comparison of the Convergence Behavior of the Methods . . . 22

6 Conclusion **23**

1 Introduction

1.1 Motivation

In many dynamical systems, multiple agents make decisions that affect the system over time. Each agent acts according to its own objectives and preferences, such as comfort, energy usage or economic performance. However, what is optimal for an individual agent is not necessarily optimal for the system as a whole. This misalignment between individual and collective objectives can lead to inefficient outcomes. Incentive design addresses this challenge by modifying the rules or signals within the system in order to influence the agents' behavior. The goal is to motivate agents to act in a way that aligns with a desired system-level outcome, even though they continue to act in their own interest. A key difficulty is that in most scenarios the leader does not know the agents' private preferences, but still needs to design incentives online based on the observed behavior of the system.

When the system dynamics are known, incentives can be designed using a control-theory-based approach, where the design relies on an explicit description of how the system evolves over time. In contrast, learning-based approaches do not require full knowledge of the system model and instead adapt incentives based on observed data [1].

Learning-based approaches can be divided into two main categories: model-based methods and model-free methods. In model-based learning, the dynamics of the system are first estimated from observations, and incentives are then computed using the estimated model. In contrast, model-free learning methods do not attempt to identify the system model explicitly, but instead adjust the decision rules directly based on observed performance.[2]

In this thesis, we study incentive design in a dynamic multi-agent setting with linear system dynamics. We consider both a model-based approach, where the system dynamics are explicitly used in the design of incentives, and a model-free reinforcement learning approach, where incentives are learned directly from observed data. The objective is to evaluate how effectively these methods guide the agents' behavior toward a desired system-level outcome, and to compare their performance with a control-theoretic benchmark.

1.2 Problem Formulation

This section formulates the incentive design problem considered in this thesis. We first introduce the general multi-agent dynamical system and describe the roles of the agents and the leader. The incentive design problem is then defined, followed by the specific two-zone thermal building model used as the main problem setup. Finally, the thesis questions are stated.

1.2.1 Multi-agent Dynamical System

A dynamical system describes how the state of a system evolves over time as a function of its current state, control input, and possible disturbances. In discrete time, such a system can be written as

$$x(t+1) = f(t, x(t), u(t), w(t)), \tag{1}$$

where $x(t) \in \mathbb{R}^n$ is the state, $u(t) \in \mathbb{R}^m$ is the control input and $w(t) \sim \mathcal{N}(0, W)$ represents stochastic disturbances. A commonly used model is the linear stochastic system

$$x(t+1) = Ax(t) + Bu(t) + d + w(t), \tag{2}$$

where A and B are system matrices, d represents a constant or slowly varying external disturbance.

To evaluate the performance of a control strategy, a cost function is introduced. In the Linear Quadratic Gaussian (LQG) framework, this is typically defined as a quadratic function of the state and control:

$$J(u) = \mathbb{E}\left[x(T)^\top Q^f x(T) + \sum_{t=0}^{T-1} \left(x(t)^\top Q x(t) + u(t)^\top R u(t)\right)\right], \tag{3}$$

The matrices Q and R determine how strongly deviations in the state and control are penalized. This formulation provides a tractable and widely used framework for optimal control under uncertainty.

In the context of multiple decision-making agents, the system may involve several control inputs, the system can then be written as

$$x(t+1) = Ax(t) + \sum_{i=1}^{m} B_i u_i(t) + d + w(t), \tag{4}$$

where $u_i(t) \in \mathbb{R}^m$ denotes the control input of agent i, and B_i represents how agent i's input affects the system. If all agents share a common objective, the individual cost functions can be replaced by a common system-level cost function,

$$J = \mathbb{E}\left[x(T)^\top Q^f x(T) + \sum_{t=0}^{T-1}\left(x(t)^\top Q x(t) + \sum_{i=1}^{m} u_i(t)^\top R_i u_i(t)\right)\right]. \tag{5}$$

This formulation can be rewritten as a standard single-agent LQG problem by defining the joint control input

$$u(t) = \begin{bmatrix} u_1(t) \\ \vdots \\ u_m(t) \end{bmatrix}, \quad B = \begin{bmatrix} B_1 & \cdots & B_m \end{bmatrix}, \quad R = \begin{bmatrix} R_1 & & 0 \\ & \ddots & \\ 0 & & R_m \end{bmatrix}. \tag{6}$$

1.2.2 Agents and the Leader

In a multi-agent dynamical system, agent i controls the control input u_i. The leader does not control any of the control inputs, but can influence the system in other ways, which will be covered later. Agents are non-cooperative and their actions are determined by minimizing their own cost function J_i. If the system consists of m agents, the cost function for agent i can be written as

$$\begin{aligned} J_i(u_1, \ldots, u_m) = \mathbb{E}\Big[&(x_i(T) - x_i^{des})^\top Q_i^f (x_i(T) - x_i^{des}) \\ &+ \sum_{t=0}^{T-1}\Big((x_i(t) - x_i^{des})^\top Q_i (x_i(t) - x_i^{des}) \\ &\quad + (u_i(t) - u_i^{des})^\top R_i (u_i(t) - u_i^{des})\Big)\Big]. \end{aligned} \tag{7}$$

The cost function of agent i depends on all control inputs $\{u_j\}_{j=1}^m$, although only u_i is explicitly expressed in the cost function. This is due to the agent's state x_i, which evolves under the influence of all control inputs together. In the cost function, the agent is penalized for deviations in the control input that differ from a desired control u_i^{des}. It is also penalized for states that differ from the desired state x_i^{des} of agent i. The desired state and control generally differ between agents, and that will therefore drive the agents to take different actions to satisfy their own cost function. The leader also has a cost function defined as

$$\begin{aligned} J_L(u_1, \ldots, u_m) = \sum_{i=1}^{m} \mathbb{E}\Big[&(x_i(T) - x_L^{des})^\top Q_i^f (x_i(T) - x_L^{des}) \\ &+ \sum_{t=0}^{T-1}\Big((x_i(t) - x_L^{des})^\top Q_i (x_i(t) - x_L^{des}) \\ &\quad + (u_i(t) - u_i^{des})^\top R_i (u_i(t) - u_i^{des})\Big)\Big]. \end{aligned} \tag{8}$$

Here it can be noted that the cost function of the leader is structurally identical to the sum of the agents' cost functions with one exception. Each agent's desired state x_i^{des} has been changed to the leader's desired state x_L^{des}. There are many other ways a leader cost function can be defined, e.g. different weighting matrices Q_i or R_i. This thesis is restricted to only differences in the desired state for the leader.

1.2.3 The Incentive Design Problem

In the multi-agent setting described above, each agent selects its control input in order to minimize its own cost function. This interaction between self-interested agents naturally leads to the concept of a Nash equilibrium. A Nash equilibrium is a set of control strategies where no agent can reduce its own cost by unilaterally changing its control input, given the strategies of the other agents. Nash equilibrium will be described more in depth in Section 2.2. At the same time the leader has a desired system-level objective. However, the leader does not directly control the system inputs, but can instead influence the agents indirectly.

The incentive design problem considered in this thesis is therefore the following: how can the leader modify the agents' decision-making, without taking control over their actions, such that the resulting system behavior aligns with a desired global objective? To achieve this, the leader introduces additional terms in the agents' cost functions. These incentive signals are designed based on the observed system behavior and aim to steer the agents toward more desirable actions, while the agents still act in their own self-interest. More specifically, the leader modifies the agents' cost functions by adding an incentive term to Equation (7). In this thesis the incentivized cost function for agent i is given by

$$J_i^{\text{inc}}(u_1, \ldots, u_m) = J_i(u_1, \ldots, u_m) + \mathbb{E}\left[\sum_{t=0}^{T-1} \rho_i(t)^\top (u_i(t) - u_i^{des})\right]. \tag{9}$$

Here, $\rho_i(t)$ represents the incentive signal designed by the leader for agent i at time t. This term introduces a linear modification of the agent's control cost, effectively shifting the agent's optimal decision without changing the system dynamics. The objective is to design $\rho_i(t)$ such that the agents, when minimizing their modified cost functions, generate system behavior that aligns with the desired objective defined by the leader's cost function. The challenge lies in determining suitable incentive signals that lead to the desired system behavior, especially in situations where the system dynamics are not fully known.

1.2.4 Problem Setup

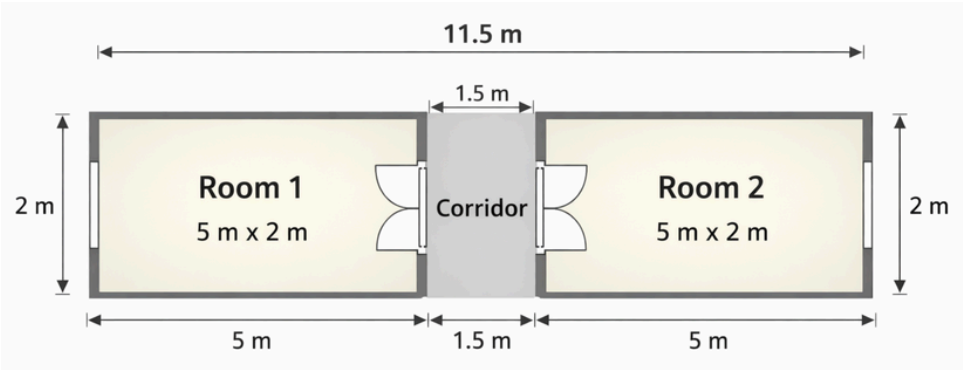


Figure 1: Geometry of the problem

For this example, the case of a building with two thermal zones is considered, as illustrated in Figure 1. The model is based on the physics-based building model presented in [3], here applied to a simplified two-zone system. A standard physics-based energy balance model for each zone is given by

$$\rho_{\text{air}} v_i C_p \frac{dT_i}{dt} = \sum_{j \in \mathcal{N}_i} h_{ij} a_{ij} (T_j - T_i) + h_{i,o} a_{i,o} (T_\infty - T_i) + \dot{m}_i C_p (T_i^s - T_i), \tag{10}$$

which describes the thermal dynamics of each zone based on an energy balance. The first term represents heat transfer between adjacent zones, the second term accounts for heat exchange with the external environment, and the third term represents the effect of the supply air.

The variables are defined as follows:

- $T_i(t)$: temperature of zone i at time t
- $T_i^s(t)$: supply-air temperature in zone i
- T_∞: outside ambient temperature
- v_i: air volume of zone i (30 m^3)
- ρ_{air}: air density (1.18 $\mathrm{kg/m}^3$)
- C_p: specific heat capacity of air (1000 $\mathrm{J/(kg{\cdot}K)}$)
- $\dot{m}_i$: mass flow rate of supply air (0.295 kg/s)
- a_{ij}: wall area between zones i and j (0 m^2)
- h_{ij}: heat transfer coefficient between zones (10.89 $\mathrm{W/(m^2{\cdot}K)}$)
- $a_{i,o}$: exterior wall area of zone i (6 m^2)
- $h_{i,o}$: exterior heat transfer coefficient (8.9 $\mathrm{W/(m^2{\cdot}K)}$)

The parameter values are taken or adapted from Coogan et al. [3]. Note that $a_{ij} = 0$ implies that no direct thermal coupling between the zones is considered in this simplified model. According to Coogan et al. [3], the model can then be rewritten on the discrete-time form, as Equation (2), where $x(t) = [T_1(t), T_2(t)]$ is the state vector of zone temperatures, and $u(t) = [T_1^s(t), T_2^s(t)]$ are the control inputs representing the supply-air temperatures in each zone.

The values obtained for the system are

$$A = \begin{bmatrix} a_1 & 0 \\ 0 & a_2 \end{bmatrix} = \begin{bmatrix} 0.9826 & 0 \\ 0 & 0.9826 \end{bmatrix}, \quad B = \begin{bmatrix} b & 0 \\ 0 & b \end{bmatrix} = \begin{bmatrix} 0.0083 & 0 \\ 0 & 0.0083 \end{bmatrix}, \quad d = \begin{bmatrix} 0.3003 \\ 0.3003 \end{bmatrix}. \tag{11}$$

Each agent's cost function can now be written as

$$J_i(u) = \mathbb{E}\left[q_i(x_i(T) - x_i^{des})^2 + \sum_{t=0}^{T-1} (q_i(x_i(t) - x_i^{des})^2 + r(u_i(t) - T_\infty)^2) \right]. \tag{12}$$

The cost function for the system is on the form

$$J(u_1, u_2) = \sum_{i=1}^{2} \mathbb{E}\left[q_i(x_i(T) - x_L^{des})^2 + \sum_{t=0}^{T-1} (q_i(x_i(t) - x_L^{des})^2 + r(u_i(t) - T_\infty)^2) \right] \tag{13}$$

with the variables defined and interpreted as

- x_1^{des}: desired temperature of zone 1 (21°C)
- x_2^{des}: desired temperature of zone 2 (24°C)
- x_L^{des}: common desired temperature for the building (26°C)
- T_∞: outside ambient temperature (33°C)
- $q_1 = 3$, $q_2 = 7$: weights on temperature deviation (comfort cost)
- $r = 1$: weight on control effort (energy cost)

The scalars q_i and r here are the one-dimensional case of the matrices Q_i and R_i in Equation (7), since each agent has a single state component and a single control input.

In this model, the desired control input u_i^{des} is identified with the outside temperature T_∞. The penalty $r(u_i(t) - T_\infty)^2$ therefore represents the cost of heating or cooling the supply air relative to the outside temperature. When incentives are introduced, each agent's cost function is modified by adding a linear incentive term in the control input. The incentivized cost function for agent i can be written as

$$J_i^{\text{inc}}(u) = \mathbb{E}\left[q_i(x_i(T) - x_i^{des})^2 + \sum_{t=0}^{T-1}\left(q_i(x_i(t) - x_i^{des})^2 + r(u_i(t) - T_\infty)^2 + \rho_i(t)(u_i(t) - T_\infty)\right)\right]. \tag{14}$$

Here, $\rho_i(t)$ is the incentive signal designed by the leader for agent i at time t. The objective is to determine incentive signals $\rho(t) = [\rho_1(t), \rho_2(t)]$ for all $t \in \{0, 1, ..., T-1\}$ to align the agents' Nash equilibrium strategies such that they minimize the cost function of the leader.

1.2.5 Thesis Questions

The goal of this thesis is to investigate how a leader can design incentives to align a multi-agent Nash equilibrium with the leader's cost function when the system is fully known, partially known, or unknown. Using the two-zone thermal problem introduced in Section 1.2.4 as a setup, we will consider three models. First, we consider the analytical solution to the problem from Coogan et al. [3], which assumes that the whole system is known. This will also serve as a reference for the other models since this is the exact analytical solution. Second, we will introduce a recursive least squares (RLS) method to estimate the system matrix A and then apply the optimal analytical solution to the estimated system. This method therefore assumes that the leader does not have knowledge of the system matrix A, but still knows the rest of the system. Third, we will consider a model-free RL approach. This approach will build on a deep deterministic policy gradient (DDPG) implementation and will not try to estimate the system, but instead train a neural network to output incentives that minimize the leader's cost function. Specifically, we address the following questions:

- Does the model-based design with known dynamics correctly recover the analytical solution from Coogan et al. [3]?
- How does the convergence of the RLS-based method to the leader's optimal cost depend on the time horizon T and noise covariance W?
- How does the convergence of the model-free method to the leader's optimal cost depend on the time horizon T?
- How does the model-free method compare to the RLS-based method in terms of error in cost and stability?
- Which method is preferable and under which conditions?

1.3 Use of AI Tools

During the project, AI tools such as Claude (Anthropic) and ChatGPT (OpenAI) were used. They were used to support understanding of the RLS and DDPG methods, with information later verified against the cited literature. Claude was also used to assist in debugging the implementation of the RLS and DDPG methods in Python, and both AI tools were used to identify grammatical errors and typos in the manuscript.

2 Theoretical Background

This chapter introduces the theoretical background used in the thesis. Sections 2.1, 2.2, and 2.3 describe the necessary background on optimal control theory. In Section 2.4, the analytical solution to the fully known system is introduced. Section 2.5 covers the model-based algorithm for the partially known system, and finally, Section 2.6 covers the model-free DDPG algorithm.

2.1 Optimal Control Problem

Given the system defined by Equation (2) and the cost function $J(u)$ defined by Equation (7), the LQG optimal control problem can be written as

$$\min_{\{u_t\}_{t=0}^{T-1}} J(u). \tag{15}$$

Expanding the quadratic terms in $J(u)$ gives

$$\begin{aligned} J(u) = \mathbb{E}\Big[& x(T)^\top Q^f x(T) - x(T)^\top (Q^f + (Q^f)^\top) x^{des} \\ & + \sum_{t=0}^{T-1} \Big(x(t)^\top Q x(t) - x(t)^\top (Q + Q^\top) x^{des} \\ & \quad + u(t)^\top R u(t) - u(t)^\top (R + R^\top) u^{des} \Big) + C \Big]. \end{aligned} \tag{16}$$

Here C represents constant terms that appear in the cost function. Since constant values will not affect the minimizer values for the cost function, they can be neglected. The linear terms can be identified, and the following coefficients are defined:

$$2q(t) = -(Q + Q^\top) x^{des} \tag{17}$$

$$2r(t) = -(R + R^\top) u^{des} \tag{18}$$

$$2q^f = -(Q^f + (Q^f)^\top) x^{des}. \tag{19}$$

Assuming (A, B) are stabilizable, the solution to the optimal control, i.e. the control that minimizes the cost function, is given by

$$u(t) = -K(t)x(t) + k(t), \quad t = 0, 1, \cdots T-1 \tag{20}$$

where $K(t)$ and $k(t)$ are given by the recursive Riccati equations [2]

$$\begin{aligned} S(T) &= Q^f, \\ K(t) &= (R + B^\top S(t+1) B)^{-1} B^\top S(t+1) A, \\ S(t) &= Q + A^\top S(t+1) A - A^\top S(t+1) B (R + B^\top S(t+1) B)^{-1} B^\top S(t+1) A \end{aligned} \tag{21}$$

and

$$\begin{aligned} s(T) &= q^f, \\ k(t) &= (R + B^\top S(t+1) B)^{-1} (B^\top s(t+1) + r(t)), \\ s(t) &= q(t) + (A - BK(t))^\top s(t+1) - K(t)^\top r(t), \quad t = T-1, ..., 0. \end{aligned} \tag{22}$$

2.2 Agent Nash Equilibrium

In multi-agent systems, each agent aims to minimize its own cost function while interacting with other agents. A Nash equilibrium is defined as a set of strategies, such that no agent can unilaterally improve its outcome by changing its own strategy. Consider a system with N agents, where each agent i selects a control sequence u_i to minimize its own cost function $J_i(u_1, \ldots, u_N)$, while the control sequences, u_j of all other agents $j \neq i$ are held fixed.

A strategy profile $(u_1^*, \ldots, u_N^*)$ is a Nash equilibrium if no agent can decrease its own cost by changing its control sequence while the control sequences of all other agents are kept fixed. More precisely, for each agent i,

$$J_i(u_1^*, \ldots, u_i^*, \ldots, u_N^*) \leq J_i(u_1^*, \ldots, u_i, \ldots, u_N^*) \tag{23}$$

for all admissible control inputs u_i.

In the context of dynamic systems, such as linear-quadratic-gaussian games, the Nash equilibrium typically corresponds to a set of feedback control laws solved with the optimal solution introduced in section 2.1. In the single-agent case, the optimal control law is obtained from the Riccati recursions in (21)–(22), where the solution depends only on the system dynamics and cost parameters. In contrast, in a multi-agent setting, each agent minimizes its own cost function while interacting with others through the shared dynamics. As a result, each agent's Riccati recursion depends on the strategies of the other agents. This leads to a system of coupled Riccati equations, where the solutions must be determined simultaneously to satisfy the Nash equilibrium conditions.[3]

2.3 Social Optimal Solution

While the optimal control problem in Equation (15) considers a single decision-maker, and the Nash equilibrium extends this framework to multiple interacting agents, it is not necessarily the strategy that maximizes the outcome for the whole system. While each agent acts optimally from its own perspective, the combined outcome may be suboptimal for the system as a whole.

In the social optimal solution, a centralized planner minimizes a global cost function that captures the performance of the entire system, rather than individual agent objectives. For a multi-agent system, this can be formulated as

$$\min_{\{u_t\}_{t=0}^{T-1}} J_{\text{social}}, \tag{24}$$

where J_{social} represents a system-level objective constructed by the planner, which may not coincide with the agents' individual preferences.

Under this formulation, the problem reduces to a standard optimal control problem, similar to Equation (15), but with a modified cost function. Consequently, the optimal control law can again be expressed in feedback form,

$$u^*(t) = -K^*(t)x(t) + k^*(t), \tag{25}$$

where $K^*(t)$ and $k^*(t)$ are obtained from Riccati recursions analogous to Equations (21)–(22), but defined with respect to the social cost.

The discrepancy between the social optimal solution and the Nash equilibrium motivates the study of incentive design, where the goal is to modify individual cost functions so that the Nash equilibrium aligns with a desired global objective [3].

2.4 Incentive Design when the Model is known

Incentive design aims to influence the agents' equilibrium behavior by modifying the weights in their objectives, i.e. their cost functions. The goal is to design these weights such that the resulting Nash equilibrium feedback gains match a prescribed target gain, taken to be the socially optimal gain. Each agent is assumed to act optimally with respect to its own cost function. For an agent cost function on the form (16), the optimal control is given by Equation (20). This will then be the control the agent is assumed to take if no incentive is added. By adding a linear incentive term $\rho(t)$ to the cost function, the modified agent cost function takes the form

$$\begin{aligned} J(u) = \mathbb{E}\Big[&(x(T) - x^{des})^\top Q^f (x(T) - x^{des}) \\ &+ \sum_{t=0}^{T-1} \Big((x(t) - x^{des})^\top Q (x(t) - x^{des}) \\ &\quad + (u(t) - u^{des})^\top R (u(t) - u^{des}) + \rho(t)^\top (u(t) - u^{des}) \Big) \Big]. \end{aligned} \tag{26}$$

By designing the incentive carefully, $\rho(t)$ can be chosen such that the agent Nash equilibrium coincides with the trajectories that minimize the team cost function.

Since constant terms in the cost function will not influence the minimizer, only the linear term $\rho(t)^\top u(t)$ from the incentive will influence the minimization. From Equation 16, it can be seen that the incentive will influence the linear control input coefficient, which will have the form

$$2r_{new}(t)^\top u(t) = (2r(t)^\top + \rho(t)^\top)u(t). \tag{27}$$

Using the Riccati formulas, the optimal feedback can be calculated as before but with $r_{new}(t)$ instead of $r(t)$. An important assumption here is that the team cost function and the agents' cost function share the same quadratic weights Q and R. Otherwise, the resulting feedback gain $K(t)$ will not match the team-optimal feedback $K^*(t)$ and the linear incentive cannot align them. The linear incentive can only match the affine term $k(t)$.

Coogan et al. [3] present an algorithm to calculate the incentive that ensures the agents' optimal feedback coincides with the team optimal feedback. First, the team-optimal feedback is calculated using the team cost function and Equations (21) and (22). This gives

$$u(t) = -K^*(t)x(t) + k^*(t) \tag{28}$$

where $K^*(t), k^*(t)$ denotes the team optimal feedback. The incentive that drives agent i from $k(t)$ to the optimal feedback $k(t)^*$ can then be calculated using the recursive equations

$$g_i(T) = 0 \tag{29}$$

$$\rho_i(t) = 2r_i u_i^{des} - 2b(g_i(t+1) + S_i(t+1)d_i) - 2k_i^*(t)(r_i + b^2 S_i(t+1)) \tag{30}$$

$$g_i(t) = -q_i x_i^{des} + (a_i - bK_i(t)^*)(g_i(t+1) + S_i(t+1)d_i) + K_i^*(t)r_i u^{des} - \frac{1}{2}K_i^*(t)\rho_i(t). \tag{31}$$

Here a_i, b are defined as in Equation (11) and r_i is defined as the value R_{ii} from the R matrix. d_i is defined as the i-th component of the disturbance vector d also defined in Equation (11). The recursive equations give a series of $\{\rho(t)\}_{t=0}^{T-1}$ such that the agents' feedback will be $k(t) = k^*(t)$ and therefore minimize the team cost function [3].

2.5 Recursive Least Squares

When the system parameters are unknown and may vary over time, they must be estimated from observed data before a model-based control or incentive design approach can be applied. A standard approach for this is recursive least squares (RLS), which is a recursive parameter estimation method for linear stochastic systems.

In general, recursive tracking algorithms can be written on the form

$$\hat{\theta}_{k+1} = \hat{\theta}_k + L_k\big(y_k - \varphi_k^\top \hat{\theta}_k\big) \tag{32}$$

where L_k is an adaptation gain, y_k is the observation, and φ_k is the regressor. In the time-varying case, the adaptation gain should not decay to zero, since the estimator must remain responsive to new observations in order to track changes in the underlying parameters. A common way of choosing the adaptation gain L_k is through the Recursive Least Squares (RLS) algorithm.

The RLS algorithm updates the parameter estimate recursively as

$$L_k = \frac{P_k\varphi_k}{\alpha + \varphi_k^\top P_k \varphi_k}, \tag{33}$$

$$\hat{\theta}_{k+1} = \hat{\theta}_k + L_k\big(y_k - \varphi_k^\top \hat{\theta}_k\big), \tag{34}$$

$$P_{k+1} = \frac{1}{\alpha}\left(P_k - L_k\varphi_k^\top P_k\right), \tag{35}$$

where the matrix P_k can be interpreted as a measure of uncertainty in the parameter estimate $\hat{\theta}_k$ and $\alpha \in (0, 1]$ is the forgetting factor. This makes RLS suitable for both constant and time-varying systems, depending on the choice of α. [4]

2.6 Deep Deterministic Policy Gradient

DDPG is a model-free reinforcement learning method designed for continuous action spaces. The learning process is formulated in discrete time, where the agent interacts with the environment in steps of the form $(s(t), a(t), r^{RL}(t), s(t+1))$. Here, $s(t)$ denotes the state at time t, $a(t)$ the action taken, $r^{RL}(t)$ the reward received, and $s(t+1)$ the subsequent state.

The algorithm consists of two main components: an actor and a critic. The actor represents the policy and can be interpreted as the decision-making component, producing actions based on the current state. The critic evaluates the quality of these actions by estimating a value function. More precisely, the actor represents a deterministic policy, denoted by $\mu_\theta(s(t))$, which maps each state $s(t)$ to a specific action,

$$a(t) = \mu_\theta(s(t))$$

The critic approximates the action-value function $Q^\mu(s(t), a(t))$, which represents the expected cumulative reward obtained by taking action $a(t)$ in state $s(t)$, and thereafter following the current policy μ. In other words, $Q^\mu(s(t), a(t))$ evaluates how good a particular action is in a given state, taking into account both the immediate reward and all future rewards under the current policy.

The DDPG algorithm learns from sampled transitions $(s(t), a(t), r^{RL}(t), s(t+1))$. The critic is trained by minimizing the difference between its current estimate and a target value given by

$$y(t) = r^{RL}(t) + \gamma Q'(s(t+1), \mu'(s(t+1))),$$

where $\gamma \in (0, 1)$ is a discount factor and Q' and μ' are target networks. These target networks are slowly updated towards the learned networks to improve stability during training.

The actor is updated using the deterministic policy gradient, which adjusts the policy parameters in a direction that increases the action-value estimated by the critic. To further stabilize learning, a replay buffer is used to store previously observed transitions. During training, mini-batches are sampled from this buffer, which reduces correlations between consecutive samples and improves data efficiency. [5]

3 Methods

The problem formulated in Section 1.2.4 is analyzed using three methods, presented in Section 3.1.1, 3.1.2 and 3.2. The first method computes the optimal incentives given that the full system is known. The second method assumes that the system matrix A is unknown and estimates it using the recursive least squares algorithm. Then the analytical solution for incentive design is applied to the estimated model. Finally, the third method directly estimates the incentives using only the feedback from applying the incentives to the system and learning from observed trajectories.

3.1 Model-based Methods

The model-based methods assume that the structure of the system dynamics and the cost functions are known. The first case considers the ideal setting where the system matrix A is known exactly, allowing the analytical incentive design formulas, described in Section 2.4, to be applied directly. The second case considers a more realistic setting where A is unknown, but where its structure is assumed and its entries are estimated using recursive least squares described in Section 2.5.

3.1.1 Incentive Design with a known Model

The problem defined in Section 1.2.4 was first solved under the assumption that the system dynamics are known. This corresponds to a simplified version of the framework presented by Coogan et al. [3]. The implementation was carried out in Python, where the incentive design algorithm presented in Section 2.4 was applied using the system parameters specified in Section 1.2.4. The time horizon was chosen as $T = 20$.

When the system matrix A is known, it follows from the analytical results in [3] that the recursive incentive design algorithm in Equations (29)–(31) should recover the optimal affine term, so that $k_i(t) = k_i^*(t)$. To

verify that the implementation was correct, the computed values of $k_i(t)$ were compared with $k_i^*(t)$ in Python. After this verification, the team cost J, calculated from Equation (26) using the incentives obtained for the known-A case, was used as a reference value for the learning-based case in the next section where A is unknown.

3.1.2 Incentive Design with an unknown Model

For a system where A is unknown, the leader cannot directly compute the incentive design as in Section 2.4. Instead, A is estimated by applying the recursive least squares (RLS) algorithm described in Section 2.5 in Python. The current estimate of A is then used in the same incentive design formulas as in the known-model case.

In this work, the unknown system matrix is assumed to have the diagonal form

$$\hat{A} = \begin{pmatrix} \hat{a}_1 & 0 \\ 0 & \hat{a}_2 \end{pmatrix}. \tag{36}$$

This corresponds to a simplified model where each state depends on its own previous value, while direct cross-coupling between the two agents through the matrix A is not included. The estimate $\hat{A}$ was initialized as the identity matrix I. Both $\hat{A}$ and the incentives ρ were then obtained using Algorithm 1 over $h = 100$ seasons. For each season h, the cost function $\hat{J}^{(h)}$ was calculated using Equation (26).

In this way, the estimate of a_i is updated across seasons. As more trajectory data is used, the estimate $\hat{A}$ is expected to improve, which allows the incentive formulas in Section 2.4 to be applied using an increasingly accurate estimate of the system dynamics.

To quantify the convergence of the estimated dynamics, the relative norm error,

$$\frac{\|\hat{A}^{(h)} - A\|}{\|A\|}, \tag{37}$$

and the relative error of the cost function,

$$\frac{\hat{J}^{(h)} - J^*}{|J^*|}, \tag{38}$$

were computed for each season h, and plotted as a function of the season index for different values of the horizon length T and the magnitude of the noise covariance W.

Algorithm 1: Learning-Based Incentive Design under Unknown Dynamics

Step 1. The leader's optimal control input with respect to the estimated system is computed from Equations (21)–(22), using the estimated values $\hat{a}_i^{(h)}$. This gives

$$\hat{K}_i^{(h)*}(t), \quad \forall t \in \{0, 1, \ldots, T-1\}, \tag{39}$$

$$\hat{k}_i^{(h)*}(t), \quad \forall t \in \{0, 1, \ldots, T-1\}. \tag{40}$$

These quantities do not correspond to the optimal control law of the true system, but rather to that of the estimated system.

Step 2. Using the optimal control law of the estimated system, the incentive estimates $\hat{\rho}_i^{(h)}$ are computed from the formulas in Section 2.4, but the true values of A, K^*, and k^* are replaced by their estimated counterparts. This gives

$$\hat{\rho}_i^{(h)}(t), \quad \forall t \in \{0, 1, \ldots, T-1\}. \tag{41}$$

Step 3. Using the estimated incentives, the open-loop Nash equilibrium trajectories are computed for both agents. These trajectories are generated according to

$$x^{(h)}(0) = 0, \tag{42}$$

$$x^{(h)}(t+1) = Ax^{(h)}(t) + Bu^{(h)}(t) + w(t), \quad t = 0, 1, \ldots, T-1. \tag{43}$$

In this step, the constant disturbance term d is not included explicitly in the dynamics, since the RLS update (see Equation (46)) estimates a model without a constant offset term. Because d is assumed to be known, its contribution can be removed before applying the RLS algorithm, without affecting the structure of the algorithm.

The control input is given by

$$u^{(h)}(t) = -K^{(h)}(t)x^{(h)}(t) + k^{(h)}(t), \tag{44}$$

where $K^{(h)}(t)$ and $k^{(h)}(t)$ are obtained from the agents' optimal responses when the estimated incentives $\hat{\rho}_i^{(h)}(t)$ are included in their cost functions.

Step 4. With these trajectories, the RLS algorithm described in Section 2.5 can be applied to update the estimate of A. The recursive update formulas are:

$$S_i(t) = \frac{P_i^{(h)}(t)x_i(t)}{\lambda + x_i^2(t)P_i^{(h)}(t)}, \tag{45}$$

$$\hat{a}_i^{(h)}(t+1) = \hat{a}_i^{(h)}(t) + S_i(t)\left(x_i(t+1) - b_1u_1(t) - b_2u_2(t) - x_i(t)\hat{a}_i^{(h)}(t)\right), \tag{46}$$

$$P_i^{(h)}(t+1) = \frac{1}{\lambda}\left(P_i^{(h)}(t) - S_i(t)x_i(t)P_i^{(h)}(t)\right), \quad t = 0, 1, \ldots, T-1. \tag{47}$$

The initial covariance matrix, P is also chosen as $P^{(0)} = I$ for the first season.

After one season, the final values $\hat{a}_i^{(h)}(T)$ and $P_i^{(h)}(T)$ are obtained. These values are then used as the initial values for the next season, such that;

$$\hat{a}_i^{(h+1)}(0) = \hat{a}_i^{(h)}(T), \tag{48}$$

$$P_i^{(h+1)}(0) = P_i^{(h)}(T). \tag{49}$$

3.2 Model-free Reinforcement Learning (DDPG)

In a model-based setting, the incentive design relies on an assumed model structure of the system dynamics and cost functions. While the exact parameters of the model may not be fully known, the design is based on this model. However, in many practical scenarios, the system model is uncertain or difficult to estimate accurately. This motivates a model-free approach, where incentives are learned directly from data without relying on an explicit model of the system dynamics.

In contrast to the model-based approach, the algorithm has no access to the system matrix and does not attempt to estimate it. Instead the incentive sequence is learned directly from the simulated trajectories using the DDPG algorithm. The DDPG algorithm was implemented in Python using the PyTorch library as the machine learning foundation.

An initial implementation defined the state as the time and the action as the corresponding incentive $\rho(t)$. However, this did not yield stable learning. This is because the reward function depends on the state $x(t)$, which in turn not only depends on t, but also on all previous incentives. Therefore, the reward was inconsistent between different instances of pairs $(t, \rho(t))$ in the replay buffer and the critic was unable to fit a consistent function. To resolve this, we instead used a degenerate case of DDPG. Instead of taking the state to be time-dependent, the state is set to a constant. The actor will then output the entire incentive sequence in a single forward pass. The critic will then map this sequence to the total cumulative reward obtained when applying the incentives to the system. This eliminates the inconsistency in the reward function since a unique sequence of incentives always generates the same expected reward.

3.2.1 Actor and Critic

In this problem, the actor network generates an incentive sequence of the form

$$(\rho(0), \rho(1), ..., \rho(T-1)). \tag{50}$$

This is done using a feedforward neural network μ_θ with two hidden layers of size 128. The input is a constant $s_1 = 1$ and the neural network outputs the full incentive sequence $\{\rho(t)\}_{t=0}^{T-1} = \{\rho_1(t), \rho_2(t)\}_{t=0}^{T-1}$, where $\rho_i(t)$ denotes the incentive for agent i at time t. The critic $Q(\{\rho(t)\}_{t=0}^{T-1})$ is also a feedforward neural network with three hidden layers of size 128. It takes the incentive sequence as input and approximates the effect on the cost function of applying the incentive sequence to the system as follows:

$$Q(\{\rho(t)\}_{t=0}^{T-1}) = \mathbb{E}\left[\sum_{k=0}^{T-1} r^{RL}(k) \Big| \{\rho(t)\}_{t=0}^{T-1} = \mu_\theta(s_1)\right]. \tag{51}$$

Here $r^{RL}(t)$ represents the reward at timestep t defined in Section 3.2.2. The critic has one more hidden layer than the actor. This follows common convention in DDPG implementations. Moreover, a target actor $\mu'_{\theta^{\mu'}}$ and a target critic $Q'_{\theta^{Q'}}$ are also used to stabilize training. These are copies of the actor and critic and are kept constant during training and then softly updated to slowly converge to the learned networks.

3.2.2 Reward Definition

The leader aims to minimize the team cost function and the stage cost can at time t be written as

$$\ell(t) = \sum_{i=1}^{2} \left[q_i(x_i(t) - x_L^{des})^2 + r(u_i(t) - T_\infty)^2\right]. \tag{52}$$

The reward is then defined as

$$r^{RL}(t) = -\ell(t) \tag{53}$$

and the total reward as

$$R^{RL} = \sum_{t=0}^{T-1} r^{RL}(t). \tag{54}$$

3.2.3 Exploration Noise

Since the DDPG algorithm is deterministic, exploration noise is here needed to ensure that the critic learns Q values away from the current policy. In this implementation, the noise is sampled from an Ornstein-Uhlenbeck process to generate correlated exploration noise, which is well-suited to dynamical systems where consecutive actions are correlated [5].

3.2.4 DDPG Updates

The learning for the neural networks is done by sampling episodes and incentives from different runs of the simulation. The training follows standard DDPG update structure and some standard DDPG variables are mapped to their counterparts and summarized in Table 1

Algorithm variable	Value in the problem	Explanation
s	1	Constant input for the actor model
a	$\{\rho(t)\}_{t=0}^{T-1}$	Incentive sequence
$r^{RL}(t)$	$r^{RL}(t)$	Reward at time t
R	R^{RL}	Total reward
$\mu(s\|\theta^{\mu})$	$\mu_\theta(s)$	Incentive output from actor neural network
$Q(s,a\|\theta^{Q})$	$Q(s,a)$	Output from critic neural network
$\mathcal{R}$	Replay buffer	Storage of sampled values of $[s,a,R]$ during training
$\mathcal{N}$	Exploration noise	Noise added to incentive during training

Table 1: Explanation of DDPG algorithm variables

The target y_i is defined as

$$y_i = R_i^{RL} \tag{55}$$

Here index i denotes the sample i.

The critic update minimizes

$$L(\theta^Q) = \frac{1}{N}\sum_{i=1}^{N}(y_i - Q(s_i,a_i|\theta^Q))^2 \tag{56}$$

and the actor is then updated by the deterministic policy gradient

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_a Q(s,a)\Big|_{s=s_i,a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)\Big|_{s=s_i} \tag{57}$$

The target networks are then updated softly by

$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'} \tag{58}$$

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'} \tag{59}$$

The training of the networks is done through episode-based updates as described in Algorithm 2

Algorithm 2: DDPG for open-loop incentive design

Randomly initialize actor $\mu(s \mid \theta^{\mu})$ and critic $Q(s, a \mid \theta^{Q})$;
Initialize target networks: $\theta^{Q'} \leftarrow \theta^{Q}, \quad \theta^{\mu'} \leftarrow \theta^{\mu}$;
Initialize replay buffer $\mathcal{R}$;
for *episode* $= 1, 2, \ldots, M$ **do**
    Generate incentive sequence $a = \mu(s_1 \mid \theta^{\mu}) + \mathcal{N}$;
    Apply a and observe agents' NE response and trajectory;
    Compute total reward R;
    Store (s_1, a, R) in $\mathcal{R}$;
    for *each gradient update step* **do**
        Sample minibatch $\{(s_1, a_i, R_i)\}$ from $\mathcal{R}$;
        Compute targets $y_i = R_i$;
        Update critic by minimizing $\frac{1}{N}\sum_i (y_i - Q(s_1, a_i))^2$;
        Update actor using deterministic policy gradient;
        Soft update: $\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}, \quad \theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'}$;
return $\{\rho^*(t)\}_{t=0}^{T-1} = \mu_{\theta^*}(s_1)$;

Here $s_1 = 1$ for all episodes.

3.2.5 Network architecture

The architecture of the networks is provided in Table 2. The optimizer used is Adam [6]. The bounding of

Actor	2 hidden layers, 128 neurons each
Critic	3 hidden layers, 128 neurons each
Activation function	ReLU
Output activation of actor	tanh bounded to $[-20, 0]$

Table 2: Network architecture for actor and critic

the actor's output serves to stabilize the training. From the analytical solution in section 3.1.1, it is known that the optimal incentives lie in the interval $[-20, 0]$ for all values of T used. This introduces some prior knowledge to the model and was used to focus learning on the relevant range. If an exact analytical solution is unavailable, a wider bound could be used, but at the cost of slower convergence.

3.2.6 Hyperparameters

The hyperparameters used in the DDPG implementation are shown in Table 3. They were chosen based on standard DDPG convention and some experimental tuning.

Hyperparameter	Value chosen	Explanation
lr_μ	10^{-5}	Learning rate for the actor network
lr_Q	10^{-4}	Learning rate for the critic network
τ	0.001	Soft update rate. Determines how softly the target network is updated in (3.2.4)
σ	0.5	Volatility coefficient. Larger σ gives noisier values
M	2000	Number of training episodes
Gradient steps	5	Number of gradient updates each episode
Minibatch size	64	Size of minibatch sampled from $\mathcal{R}$ each gradient step

Table 3: Hyperparameters for the DDPG algorithm

No discount factor γ is used in the implementation since the critic predicts the total reward each update

rather than a discounted reward.

4 Results

The evaluation was carried out in several steps. First, the model-based incentive design method was verified by applying it to a known model, where the analytical solution is available. The performance of the different methods was then assessed by comparing the obtained cost functions, denoted by $\hat{J}$, with the analytical reference cost J^*. For the RLS-based method, the estimated system matrix $\hat{A}$ was also compared with the true system matrix A. For the DDPG-based method, the learned incentives were further compared with the incentives obtained from the analytical solution.

4.1 Incentive Design when Model is known

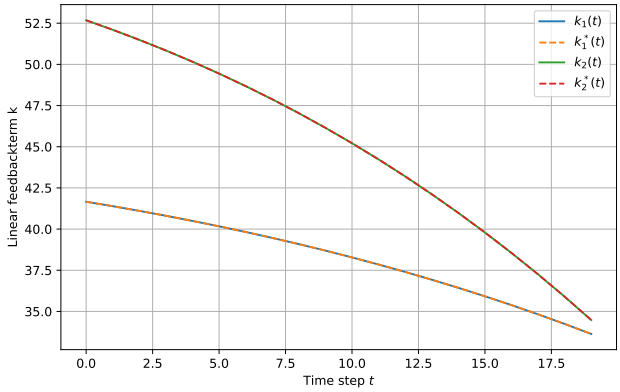


Figure 2: Comparison between the optimal system-level feedback gains $k_i^*(t)$ and the agent-level feedback gains $k_i(t)$ obtained after incentive design, when $T = 20$, assuming a known system matrix A.

The mean value of the relative error between the computed and desired feedback terms is given by

$$\frac{1}{T}\sum_{t=1}^{T}\frac{\|k(t)-k^*(t)\|}{\|k^*(t)\|} = 1.29 \times 10^{-16}.$$

This is illustrated in Figure 2 where we can see that the plot of the estimated value of the linear feedback term completely overlaps with the plot of its analytical counterpart.

4.2 Incentive Design when Model is unknown

Figure 3 shows that the cost function converges rapidly when using the RLS-based method. The relative error of the cost function decreases to below 10^{-8} after approximately 40 seasons. The shaded region indicates that the variation across random seeds is small.

Figure 4 shows that the RLS estimation of the system matrix A converges faster as the time horizon T is increased. In contrast, Figure 5 shows that the cost function of the model estimated using RLS converges more slowly for larger values of T.

Figures 6 and 7 show that convergence is achieved for all tested noise levels. However, as the noise level increases, both the system matrix estimate and the cost function exhibit slower convergence and larger steady-state errors.

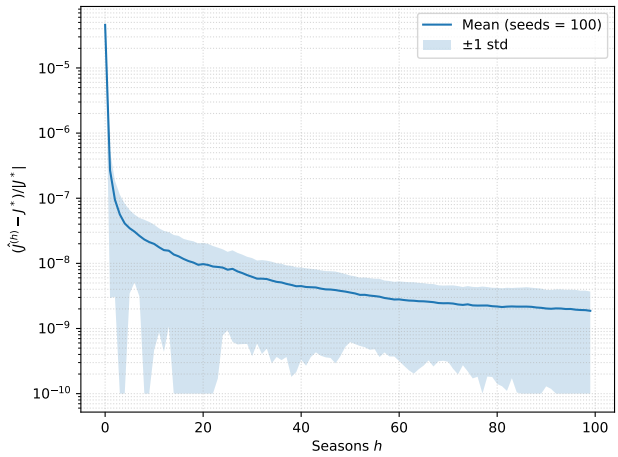


Figure 3: Relative error of the cost function, $(\hat{J}^{(h)} - J^*)/|J^*|$ as a function of the season index h. Averaged over 100 simulations, with noise covariance $W = 0.01\,I$ and time horizon $T = 20$. The results are shown on a logarithmic scale (y-axis). One standard deviation is plotted.

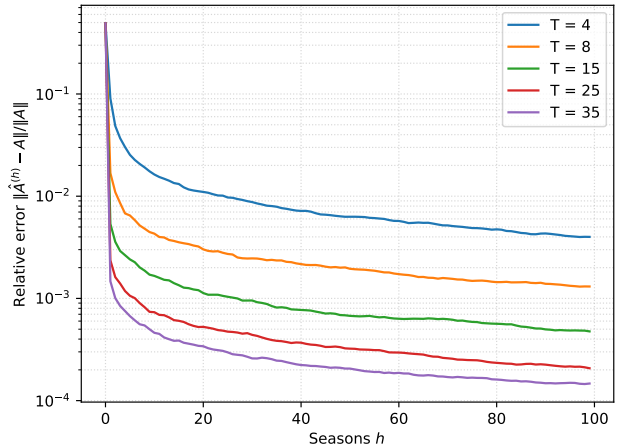


Figure 4: Relative error of the A-matrix norm, $\|\hat{A}^{(h)} - A\|/\|A\|$, as a function of the season index h. Plotted for different values of T, averaged over 100 independent simulations, with noise covariance $W = 0.01\,I$. The results are shown on a logarithmic scale (y-axis).

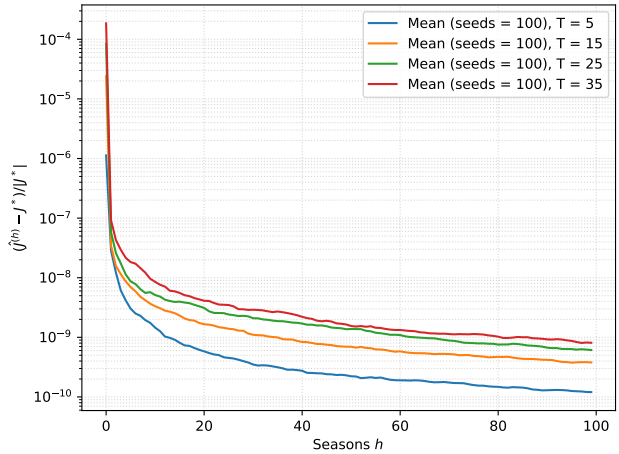


Figure 5: Relative error of the cost function, $(\hat{J}^{(h)} - J^*)/|J^*|$ as a function of the season index h. Plotted for different values of T. Averaged over 100 simulations, with noise covariance $W = 0.01\, I$. The results are shown on a logarithmic scale (y-axis).

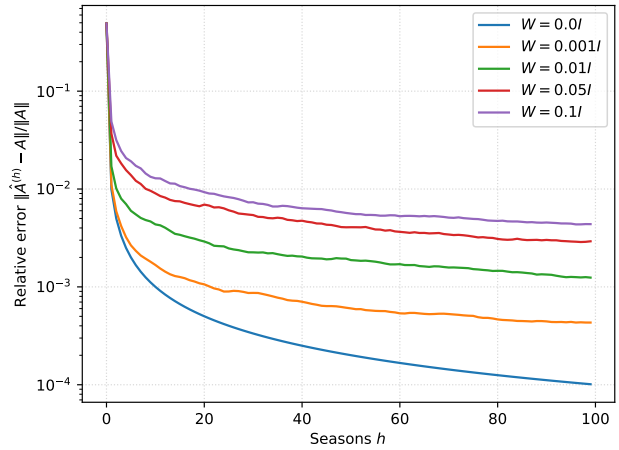


Figure 6: Relative error of the A-matrix norm, $\|\hat{A}^{(h)} - A\|/\|A\|$, as a function of the season index h, for different values of W. Averaged over 100 simulations, with $T = 8$. The results are shown on a logarithmic scale (y-axis).

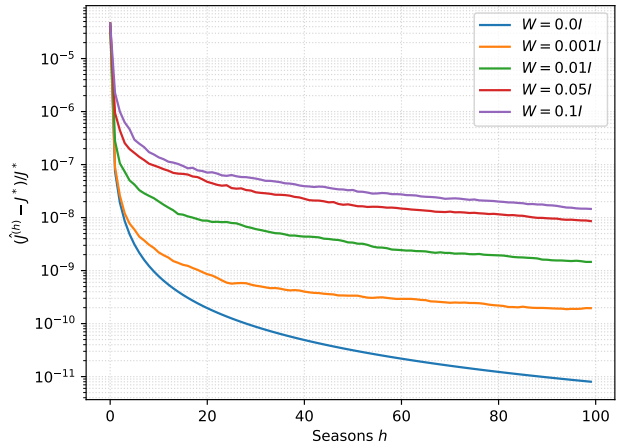


Figure 7: Relative error of the cost function, $(\hat{J}^{(h)} - J^*)/|J^*|$, as a function of the season index h. Plotted for different values of W, averaged over 100 simulations, with $T = 8$. The results are shown on a logarithmic scale (y-axis).

4.3 Model-free Incentive Design

Figure 8 shows the mean relative cost error over 5 seeds for the DDPG algorithm over 2000 episodes with one plotted standard deviation. The time horizon is set to $T = 20$ and there is no system noise. The relative error decreases for the first few hundred episodes to a minimum of around $2 \cdot 10^{-4}$. Then the error increases and seems to plateau at around $3.5 \cdot 10^{-4}$.

Figure 9 shows the mean relative cost error over 5 seeds for different time horizons T, again without system noise. The graph shows larger T converges slower than smaller T.

Figure 10 shows the relative cost error for one run of the DDPG algorithm, and Figures 11 and 12 show the corresponding learned incentives after $0, 250, 1000$, and 2000 episodes. The optimal incentives (red curve) are also plotted in the figures.

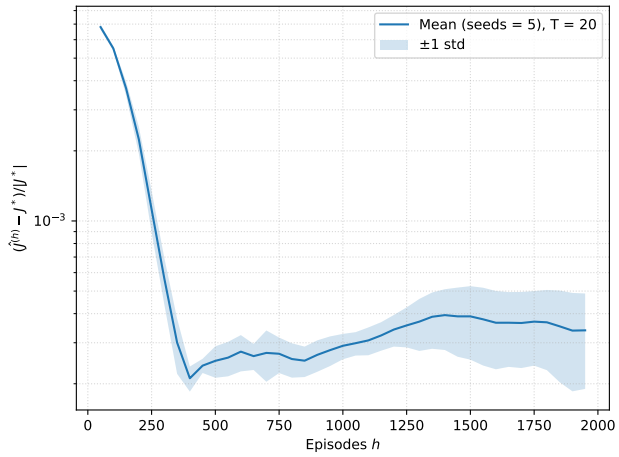


Figure 8: Mean relative error over 5 seeds of the cost function for different episodes of the DDPG algorithm. Noise covariance $W = 0$. One standard deviation is plotted and $T = 20$. The results are shown on a logarithmic scale (y-axis)

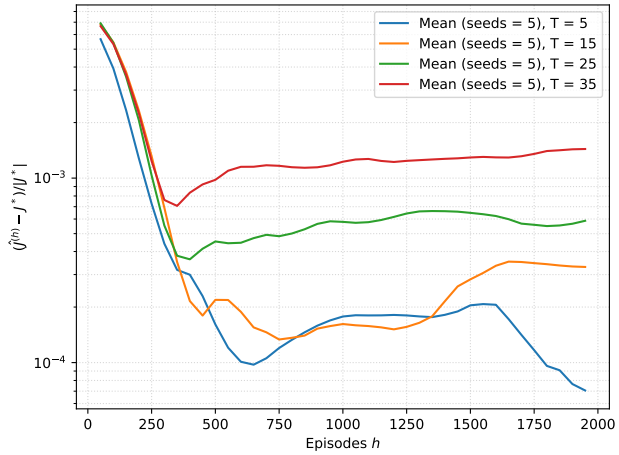


Figure 9: Relative error of the cost function for different episodes of the DDPG algorithm for different time horizons T. The results are shown on a logarithmic scale (y-axis)

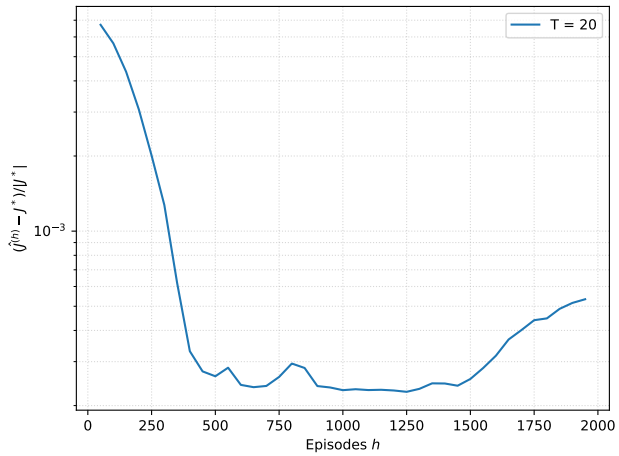
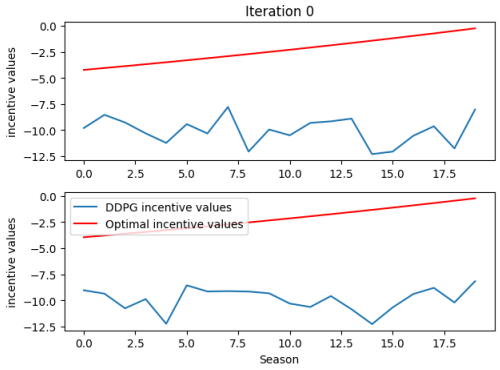
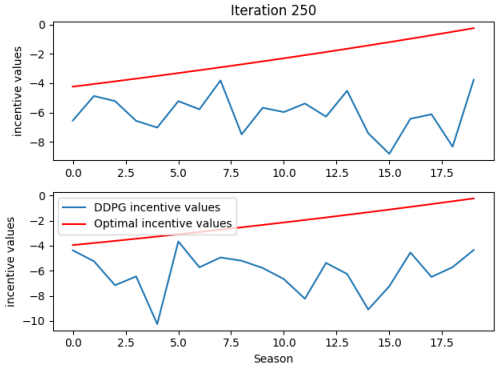


Figure 10: Relative cost error for one run of the DDPG algorithm. Time horizon $T = 20$

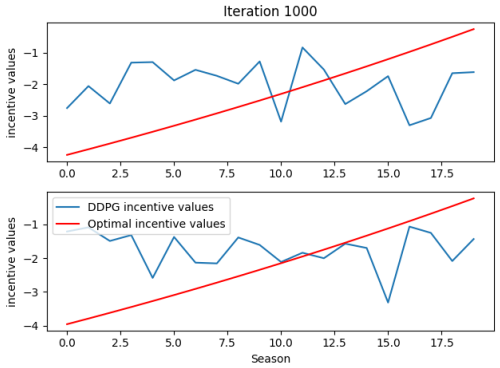


(a) Initialized incentives for the DDPG algorithm

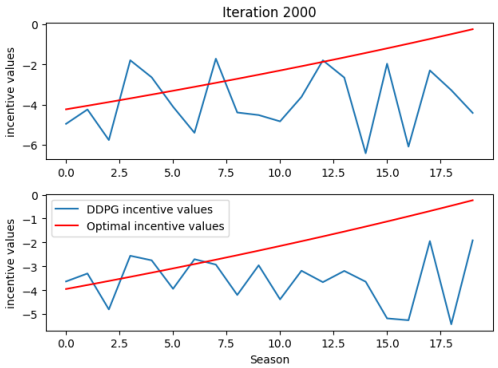


(b) Learned incentives for one run of the DDPG algorithm after 250 episodes

Figure 11: Learned incentives of one run of the DDPG algorithm. The bottom graph shows the incentive for agent 2 and the upper graph shows the incentive for agent 1. The red curve shows the optimal incentive design.



(a) Learned incentives for one run of the DDPG algorithm after 1000 episodes



(b) Learned incentives for one run of the DDPG algorithm after 2000 episodes

Figure 12: Learned incentives of one run of the DDPG algorithm. The bottom graph shows the incentive for agent 2 and the upper graph shows the incentive for agent 1. The red curve shows the optimal incentive design.

5 Analysis

In this section, the results presented in Section 4 are analyzed. In Section 5.1, it is verified that the model-based method yields the analytical solution if the system matrix A is known. Section 5.2 covers the convergence behavior of the model-based RLS method and Section 5.3 covers the robustness of the RLS method for different noise levels. This is done by examining both the error in the system matrix $\hat{A}$ and the relative cost error. Section 5.4 covers the convergence behavior of the model-free DDPG approach. Finally, Section 5.5 compares the two methods with respect to the relative cost error convergence and the information available to each method.

5.1 Model Verification

As a first step, the implementation of the incentive design framework was verified in the case where the system matrix A is known. In this setting, the analytical results presented in [3] show that the designed incentive signals should yield feedback terms $k_i(t)$ that coincide with the desired terms $k_i^*(t)$. To confirm

that the implementation is correct, the relative difference between the computed and desired feedback terms was evaluated. The relative error was found to be on the order of 10^{-16}, which corresponds to machine precision. This indicates that the two quantities are numerically identical. This result is also illustrated in Figure 2, where the trajectories of $k_i(t)$ and $k_i^*(t)$ are shown to overlap completely.

The agreement between the analytical solution and the numerical implementation confirms that the incentive design method has been correctly implemented. This provides a reliable foundation for extending the approach to the case where the system dynamics are unknown.

5.2 Convergence Behavior of the RLS Method

Figure 5 shows that larger values of T converge more slowly to the same relative cost error than smaller values of T. The error in the estimate of A instead increases for smaller T in the same season. This can be explained by the model having more information to update the estimate of A with each season, which will bring it closer to the true A. Equation (46) shows that the estimate of A is updated T times each season, and therefore, larger T leads to faster convergence, which Figure 4 shows. The question remains why the cost error still increases for larger T, even though the estimate of A is smaller for larger T at the same season. This can be explained by analyzing recursive Riccati equations (21) and (22). In the Riccati equation, each backward step involves multiplying by A. The error of A therefore propagates through the recursion and affects the computation of the feedback $K^*(t)$, $k^*(t)$. The calculated incentives thus deviate from the optimal incentives. Since the cost is computed as a sum over T trajectories, the trajectory errors will accumulate over all T steps and contribute to a larger total cost error.

5.3 Robustness of the RLS Method

To evaluate the robustness of the learning-based incentive design method, simulations were performed for different noise levels in the disturbance covariance matrix W. The results are presented in Figures 6 and 7, where the relative error of the system matrix and the cost function are shown, respectively.

From Figure 7, it can be observed that the relative cost error converges for all tested values of W. This indicates that the method remains stable even in the presence of stochastic disturbances. However, increasing the noise level leads to both slower convergence and a higher steady-state error. In particular, for larger values of W, the cost does not converge to the optimal value J^*, but instead stabilizes at a higher level. A similar trend is observed in Figure 6, where the relative error $\|\hat{A}^{(h)} - A\|/\|A\|$ is shown. While the estimate $\hat{A}^{(h)}$ converges for all noise levels, the accuracy of the estimate deteriorates as the noise increases. This demonstrates that the system identification step is sensitive to disturbances.

The observed behavior can be explained by analyzing how errors propagate through Algorithm 1. In Step 4, the system matrix $\hat{A}$ is estimated using the RLS algorithm based on noisy state trajectories. As the disturbance $w(t)$ increases, the estimation error in $\hat{A}$ also increases. In Step 1, the feedback gains $\hat{K}^{(h)*}(t)$ and $\hat{k}^{(h)*}(t)$ are computed using the estimated matrix $\hat{A}$. Since these gains are optimal only with respect to the estimated system, they are generally suboptimal for the true system. When applied to the true dynamics in Step 3, this results in suboptimal state trajectories. As the cost function depends on both the state and control input, these deviations from the optimal trajectories directly lead to an increased cost. Therefore, the degradation in performance can be traced back to the estimation error in $\hat{A}$, which affects the computed control law and leads to suboptimal trajectories, ultimately resulting in an increased cost.

In summary, the method is robust in the sense that convergence is achieved for all tested noise levels. However, the performance degrades as the noise increases, primarily due to reduced accuracy in the estimation of the system dynamics.

5.4 Convergence Behavior of Model-free Approach

Figure 8 shows that the DDPG algorithm exhibits non-monotonic convergence behavior with a steep decline in the cost error followed by a slight upward drift to a plateau. This is consistent with known instabilities of DDPG. Fujimoto et al. [7] identify several such mechanisms that lead to training instability and convergence

error. In particular, a common problem with actor-critic models is overestimation bias in the critic, where the noise will introduce a persistent overestimation. There are many more factors that can cause DDPG to become unstable and the reasons are largely still poorly understood. Matheron et al. [8] show multiple such failures when the algorithm can get stuck. The DDPG algorithm is also sensitive to the choice of hyperparameters, and poor choices can cause training to become unstable. Although the error of the cost increases slightly after around 400 episodes, the error is still relatively small and the cost function is approximately 0.035% above the optimal cost. This still implies that the model has learned a near-optimal policy. The reason for the non-monotonic behavior of the model for later episodes is probably because of the instability of the DDPG method, e.g. overestimation bias or hyperparameter sensitivity, but would require more analysis to diagnose exactly.

To further understand the convergence of the DDPG algorithm, Figures 11 and 12 can be studied. These show the incentives that the actor produces after $0, 250, 1000$, and 2000 episodes. These learned incentives correspond to the cost function in Figure 10. Figure 11a shows the initial incentives produced by the randomly initialized actor network. Figure 11b shows that the actor outputs incentive values closer to 0 and the cost function has decreased. At 1000 episodes, Figure 12a shows that the actor continues to produce incentives closer to 0 than in earlier episodes. Here, the incentives at $t < 10$ are larger than the optimal and most incentives at $t > 10$ are smaller than the optimal. The cost function also indicates that the algorithm may be stuck in a local minimum. The algorithm can then have a hard time finding the optimal since, for example, if the actor tries an incentive closer to the optimal in the early part of the sequence, the critic will probably give a worse reward since the incentives smaller than the optimal after $t = 10$ could compensate for the incentives larger than the optimal in the beginning. After 2000 episodes, Figure 12b shows that the incentives have decreased, and are now mostly less than the optimal incentives. The cost function after 2000 episodes also shows that the cost has increased and is worse than the cost between episodes 500 and 1500. This could be a symptom of DDPG instabilities such as overestimation bias or poor choice of hyperparameters, but to clarify this, further analysis of the actor and critic outputs would need to be done.

Figure 9 shows the relative cost error of the DDPG algorithm for different time horizons T over episodes. The curves for each time horizon are the mean of five seeds. The figure clearly shows that the error decreases for smaller T. This is expected since the output dimension of the actor grows linearly with T. The actor and critic must then approximate a higher-dimensional function which will generally require more samples to learn well. The cost is also the sum of T stage costs, which are then influenced by all incentives in earlier times. Thus, the error accumulates over the cost function. Combined with the DDPG instabilities discussed above, these effects imply that the algorithm needs more episodes to converge and has a higher risk of getting stuck in a suboptimal local minimum.

5.5 Comparison of the Convergence Behavior of the Methods

Comparing the convergence for the two methods with regard to error after seasons/episodes, it is clear that the model-based RLS method has an error several orders of magnitude smaller than the model-free DDPG method. The best relative error for the DDPG algorithm over 2000 episodes was around $2 \cdot 10^{-4}$ at 400 episodes, while the RLS reached a minimum relative error of around $2 \cdot 10^{-9}$ at 100 seasons. The RLS method also exhibits stable convergence, while the DDPG method did not show further improvement after 400 episodes. The error of the DDPG method was therefore around five orders of magnitude larger than the error of the RLS method and it is thus safe to declare the RLS method superior in this problem.

The difference in the errors of the methods is substantial but a fair interpretation requires accounting for the difference in information available to each method. The model-based algorithm assumes knowledge of the whole structure of the system. It uses the known linear structure of the dynamics, quadratic cost function, known input matrix B and disturbance d as well as the analytical optimal incentive formula from Coogan et al. [3] to determine the optimal incentive design. Only the system matrix A is here unknown. Meanwhile, the model-free algorithm uses much less prior knowledge of the system. The output is bounded to $[-20, 0]$ providing some structural information but other than that, it assumes no structure of the system and only treats the problem as a black box. It outputs an incentive design and then observes its impact on the rewards and then updates the incentives directly according to the DDPG algorithm. It therefore does not exploit any of the system's known dynamics and the performance gap therefore represents the difference in prior

knowledge of the system, not general superiority of the RLS method.

The conclusion that can be given here is that for a problem where the system structure is known and all but the system matrix A is known, then the RLS method will be a far better choice of model than the DDPG method. However, in other problem settings where linear models or quadratic cost cannot be assumed, the RLS method would not be applicable, while the model-free approach would remain a viable method. Whether the DDPG method would achieve good performance on other systems is, however, a separate question that lies outside the scope of this thesis.

6 Conclusion

This study has analyzed and compared the performance of different methods of a simplified version of the problem presented in [3]. When the system dynamics were known, the model-based approach successfully recovered the analytical solution. When the structure of the system dynamics was known but the system matrix A was unknown, the RLS algorithm converged very well to the true solution. The convergence of the RLS-based method was shown to depend on both the time horizon T and the noise covariance W. A larger time horizon provides more data per season, leading to faster convergence of the system matrix estimate $\hat{A}$. However, the relative cost error increases with larger T, since the accumulated error in the cost function increases with more data. Regarding noise, the method remains stable for all tested values of W, but higher noise levels result in slower convergence and a higher steady-state cost error due to reduced estimation accuracy. For the model-free approach, the DDPG algorithm demonstrated relatively fast initial learning, with a rapid decrease in cost error during the first few hundred episodes. However, the convergence was non-monotonic and less stable, with the error increasing slightly after reaching its minimum and then stabilizing. The performance tends to degrade as the time horizon T increases.

The comparison shows that the RLS-based method achieves substantially lower cost errors, converges in significantly fewer seasons, and exhibits more stable convergence than the model-free DDPG approach. However, this performance gap must be interpreted in light of the available information. The RLS-based method exploits strong prior knowledge about the system structure, including linear dynamics and quadratic costs, whereas the model-free approach operates without such assumptions. As a result, the superior performance of the RLS method reflects its use of structural information rather than an inherent advantage in all settings.

In conclusion, the RLS-based method is preferable when the system structure is known and only limited parameters need to be estimated, as it provides high accuracy and stable convergence. In contrast, the model-free approach is more suitable in scenarios where the system dynamics are unknown or cannot be accurately modeled, despite its lower accuracy and potential instability.

References

1. Ratliff LJ, Dong R, Sekar S, and Fiez T. A Perspective on Incentive Design: Challenges and Opportunities. Annual Review of Control, Robotics, and Autonomous Systems 2019; 2. DOI: 10.1146/annurev-control-053018-023634
2. Fazel M, Ge R, Kakade SM, and Mesbahi M. Global Convergence of Policy Gradient Methods for the Linear Quadratic Regulator. arXiv 2018. DOI: 10.48550/arXiv.1801.05039
3. Coogan S, Ratliff LJ, Calderone D, Tomlin C, and Sastry SS. Energy Management via Pricing in LQ Dynamic Games. *2013 IEEE 52nd Annual Conference on Decision and Control (CDC)*. 2013. DOI: 10.1109/CDC.2013.6760789
4. Guo L. Stability of Recursive Stochastic Tracking Algorithms*. Society for Industrial and Applied Mathematics 1994
5. Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, et al. Continuous Control with Deep Reinforcement Learning. arXiv preprint arXiv:1509.02971 2015. Available from: https://arxiv.org/pdf/1509.02971
6. Kingma DP and Ba J. Adam: A Method for Stochastic Optimization. *3rd International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6980 [cs.LG]
7. Fujimoto S, Hoof H van, and Meger D. Addressing Function Approximation Error in Actor-Critic Methods. Stockholm, Sweden: PMLR, 2018. Available from: https://proceedings.mlr.press/v80/fujimoto18a.html
8. Matheron G, Perrin N, and Sigaud O. THE PROBLEM WITH DDPG: UNDERSTANDING FAILURES IN DETERMINISTIC ENVIRONMENTS WITH SPARSE REWARDS. 2019. Available from: https://arxiv.org/pdf/1911.11679

TRITA – XXX-XXX 20XX:XX
Stockholm, Sverige 2026

www.kth.se